NAZAR — System Overview

Next-gen AI for Zero-defect Assurance & Root-cause analysis. Har part pe nazar, har defect ki wajah.

The problem (INDUX 5.0, PS-04)

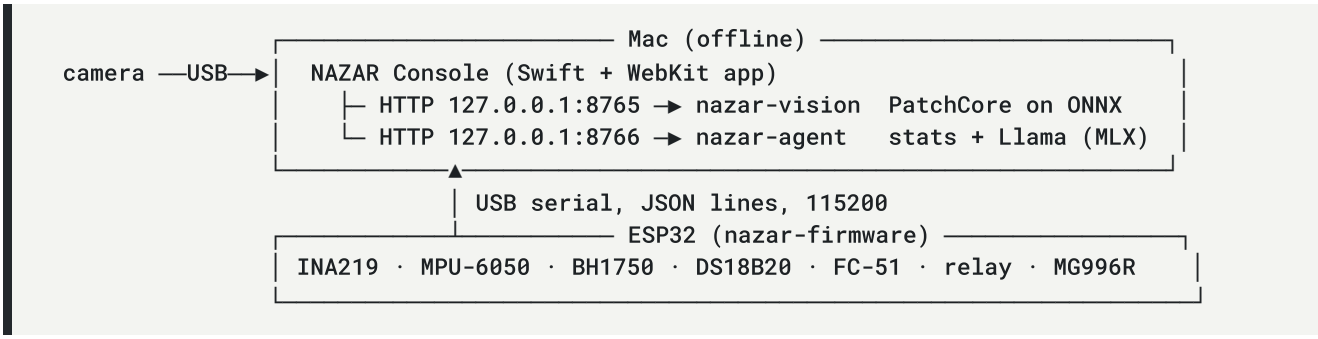
Edge-AI multi-sensor quality inspection for real-time manufacturing. Small factories inspect parts by eye. People get tired, miss defects, and when defects rise nobody can say *why* — was it the machine, the light, the material? By the time someone finds out, a whole batch is scrap.

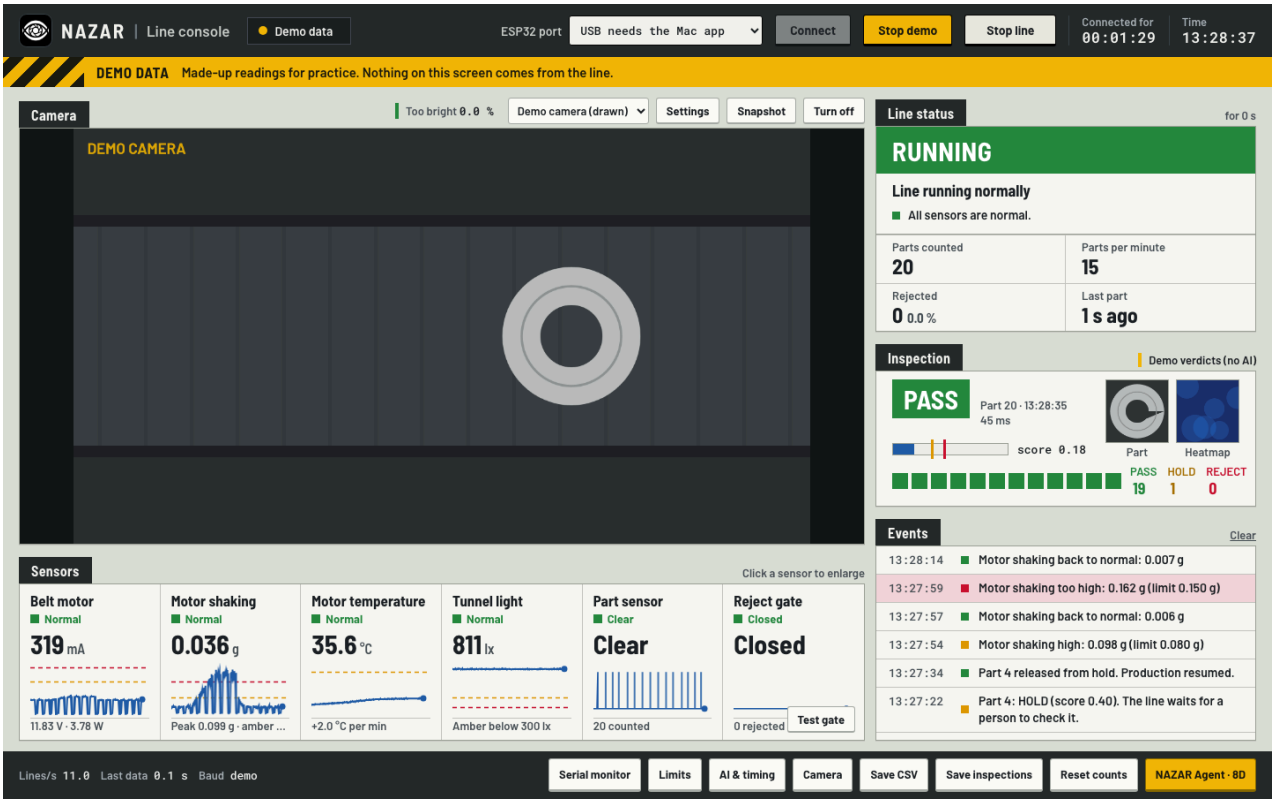
What NAZAR does

1. **Sees every part.** A camera photographs each washer as it stops under a light tunnel. An anomaly-detection model, trained only on good washers, answers **PASS**, **HOLD** (unsure — a person checks) or **REJECT**.
2. **Acts on it.** A rejected washer is kicked off the belt by a servo gate. An unsure one stops the line until the operator presses *Resume*.
3. **Watches the machine at the same time.** Vibration, motor current, motor temperature and tunnel light are logged with every inspection.
4. **Finds the cause.** A root-cause agent compares the defect rate when the machine is normal vs abnormal ("rejects are 6× higher while the motor shakes"), proves it statistically, and drafts an **8D report** in English with a local language model.
5. **Proves the fix.** After the team fixes the cause, NAZAR compares before vs after.

Everything runs **on one laptop, offline**. No cloud, no internet, no data leaves the factory.

Architecture





One washer, step by step

Time	What happens	Who
0 ms	Washer reaches the IR sensor	ESP32 sends `part`
+150 ms	Belt stops; washer centred under the camera	ESP32 relay, sends `inspect`
+450 ms	App grabs a camera frame after the picture settles	Console
+500 ms	AI scores the washer and returns PASS / HOLD / REJECT + heatmap	Vision service
+500 ms	App sends the verdict back	Console → `PASS` / `FAIL` / `HOLD`
+1.2 s	Belt restarts (HOLD: waits for the operator)	ESP32
+2 s	A REJECT reaches the gate; the arm kicks it off	ESP32 servo

Every verdict is stored with the sensor readings of that moment — that log is what the root-cause agent analyses. If the AI is off or slow, the line never waits: the ESP32 times out and the part passes, and the console logs it.

The five technologies (one repo and one PDF each)

Repo	What	Guide
nazar-hardware	Power, wiring, sensors, conveyor, gate, bring-up order	NAZAR-00-Hardware-Bringup.pdf
nazar-firmware	ESP32 firmware: sensors at 10 Hz, stop-and-inspect state machine, gate timing; one test sketch per sensor	NAZAR-01-Firmware.pdf
nazar-vision	Washer finding (OpenCV) + PatchCore anomaly model (WideResNet-50 features on ONNX Runtime) as a local service	NAZAR-02-Vision-AI.pdf
nazar-agent	p-chart, stratification with Fisher's exact test, before/after, 8D report written with Llama 3.1 8B on Apple MLX	NAZAR-03-Root-Cause-Agent.pdf
nazar-console	The operator Mac app: live line, camera, verdicts, alarms, 8D view	NAZAR-04-Console-App.pdf

How the parts talk is fixed in **CONTRACT.md** in this repo.

Run it (event day)

1. **Power and firmware.** Rig wired as in the hardware guide; flash `nazar-firmware/nazar_line`. Close the Arduino serial monitor.
2. **Start the AI.** Double-click `start-nazar.command` in this repo (starts vision and agent services and opens the console). Or start `Start Vision.command` and `Start Agent.command` yourself.
3. **Console.** Pick the ESP32 port (`/dev/cu.usbserial-…`), Connect. The band turns **LIVE DATA**. Run the belt 10 s, then *Limits → Learn normal*.
4. **Camera.** Pick the camera, open *Settings*, switch off auto exposure, lower it until the brightness meter is near 0 %.
5. **Run washers.** Each one stops, gets a verdict, and goes on. Rejects get kicked.

Demo story for the judges (about 4 minutes)

1. Run 15–20 good washers: steady PASS, quiet sensors.
2. Put in two damaged washers: REJECT with a heatmap on the damage; the gate kicks them off. One borderline part: HOLD, the line stops, operator checks, *Resume*.
3. **Loosen the motor mount.** Vibration tile goes amber/red; rejects and holds rise.
4. Open **NAZAR Agent · 8D**: the agent shows rejects are several times higher while the motor shakes, with the p-value, and drafts the 8D report.
5. Tighten the mount, press **Mark fix applied**, run more washers: the before/after table proves the fix.

Team Parakh

Shubham Mishra (lead, ECE), Roshan Kumar Singh (CSE), Purvi (CSE) · INDUX 5.0, Vikrant University Gwalior, 2–3 Nov 2026.

Open-source libraries used (with thanks): ONNX Runtime, OpenCV, NumPy, Pillow, PyTorch/torchvision (only to export the backbone once), Apple MLX + mlx-lm, Meta Llama 3.1 8B Instruct (Llama 3.1 Community License), OneWire, DallasTemperature, Barlow and Roboto Mono fonts (OFL / Apache 2.0).